

Project 1 - Part 4

Test Evidence, Bugs, Customer Questions and Assumptions

Branson Zemaitis

SIMS — Version 4.0 | COP 3515

Regression Testing

Tests 1-2 (Alice Johnson, spec Example 1): entered valid student info and saved it. Matches the spec's example exactly.

```
-----  
Student Information Management System  
Version 4.0
```

```
Programmer: Branson Zemaitis
```

```
Welcome to SIMS  
-----
```

```
Enter Student ID: 1234567  
Enter Student Name: Alice Johnson  
Enter Current GPA: 3.84
```

```
Enter Course 1 Grade: 95  
Enter Course 2 Grade: 88  
Enter Course 3 Grade: 91  
Enter Course 4 Grade: 84  
Enter Course 5 Grade: 100
```

```
Student Summary
```

```
Student ID : 1234567  
Student Name : Alice Johnson  
Current GPA : 3.84  
Academic Standing : Honors  
-----
```

```
Course Grades
```

```
Course 1 : 95  
Course 2 : 88  
Course 3 : 91  
Course 4 : 84  
Course 5 : 100  
-----
```

```
Average Grade : 91.60  
Highest Grade : 100  
Lowest Grade : 84
```

```
Student information successfully saved.
```

```
Reading student information...  
File successfully loaded.
```

```
Recovered Student Record  
-----
```

Tests 3-4 (continued): read the file back, showed the Recovered Student Record, and verified it matches what I entered.

```
Student Information Management System
Version 4.0
-----
```

```
Student ID : 1234567
Student Name : Alice Johnson
Current GPA : 3.84
Academic Standing : Honors
-----
```

Course Grades

```
Course 1 : 95
Course 2 : 88
Course 3 : 91
Course 4 : 84
Course 5 : 100
-----
```

```
Average Grade : 91.60
Highest Grade : 100
Lowest Grade : 84
-----
```

```
Verification: Recovered data matches the original data entered.
```

```
student@0193210b33c3:/workspace$
```

Saved File Contents

student_records.txt after saving (cat output) — matches the spec's Example 2, line for line.

```
===== student_records.txt (cat output) =====  
1234567  
Alice Johnson  
3.84  
Honors  
95  
88  
91  
84  
100
```

Test 5: tried to read a file I couldn't open (revoked read permission after saving). Gets the exact error message the spec's Example 3 asks for.

```
===== TEST5_FILE_ERROR =====
-----
Student Information Management System
Version 4.0

Programmer: Branson Zemaitis

Welcome to SIMS
-----

Enter Student ID: Enter Student Name: Enter Current GPA:
Enter Course 1 Grade: Enter Course 2 Grade: Enter Course 3 Grade: Enter Course 4 Grade: Enter Course 5 Grade:
Student Summary

Student ID : 1234567
Student Name : Alice Johnson
Current GPA : 3.84
Academic Standing : Honors
-----
Course Grades

Course 1 : 95
Course 2 : 88
Course 3 : 91
Course 4 : 84
Course 5 : 100

-----
Average Grade : 91.60
Highest Grade : 100
Lowest Grade : 84

Student information successfully saved.

Reading student information...

ERROR
Unable to open student_records.txt
Please verify that the file exists and that you have permission to access it.
```

Extra Regression Cases (Parts 1-3)

The spec only requires the five tests above. I also reran four cases from Parts 2 and 3 to make sure nothing broke.

Michael Brown — Good Standing, avg 79.00, high 91, low 68.

```

===== MICHAEL_BROWN_REGRESSION =====
-----
Student Information Management System
Version 4.0

Programmer: Branson Zemaitis

Welcome to SIMS
-----

Enter Student ID: Enter Student Name: Enter Current GPA:
Enter Course 1 Grade: Enter Course 2 Grade: Enter Course 3 Grade: Enter Course 4 Grade: Enter Course 5 Grade:
Student Summary

Student ID : 7654321
Student Name : Michael Brown
Current GPA : 2.91
Academic Standing : Good Standing
-----

Course Grades

Course 1 : 68
Course 2 : 91
Course 3 : 75
Course 4 : 93
Course 5 : 68

-----

Average Grade : 79.00
Highest Grade : 93
Lowest Grade : 68

Student information successfully saved.

Reading student information...
File successfully loaded.

Recovered Student Record
-----

Student Information Management System
Version 4.0
-----

Student ID : 7654321
Student Name : Michael Brown
Current GPA : 2.91
Academic Standing : Good Standing
-----

Course Grades

Course 1 : 68
Course 2 : 91
Course 3 : 75
Course 4 : 93
Course 5 : 68

-----

Average Grade : 79.00
Highest Grade : 93
Lowest Grade : 68

-----

Verification: Recovered data matches the original data entered.

```

Christopher Williams — Honors, perfect scores.

```

===== CHRISTOPHER_WILLIAMS_REGRESSION =====
-----
Student Information Management System
Version 4.0

Programmer: Branson Zemaitis

Welcome to SIMS
-----

Enter Student ID: Enter Student Name: Enter Current GPA:
Enter Course 1 Grade: Enter Course 2 Grade: Enter Course 3 Grade: Enter Course 4 Grade: Enter Course 5 Grade:
Student Summary

Student ID : 1111111
Student Name : Christopher Williams
Current GPA : 4.00
Academic Standing : Honors
-----
Course Grades

Course 1 : 100
Course 2 : 100
Course 3 : 100
Course 4 : 100
Course 5 : 100

-----
Average Grade : 100.00
Highest Grade : 100
Lowest Grade : 100

Student information successfully saved.

Reading student information...
File successfully loaded.

Recovered Student Record
-----

Student Information Management System
Version 4.0
-----

Student ID : 1111111
Student Name : Christopher Williams
Current GPA : 4.00
Academic Standing : Honors
-----
Course Grades

Course 1 : 100
Course 2 : 100
Course 3 : 100
Course 4 : 100
Course 5 : 100

-----
Average Grade : 100.00
Highest Grade : 100
Lowest Grade : 100

-----
Verification: Recovered data matches the original data entered.

```

Invalid GPA — 4.25 correctly rejected before any file save is attempted.

```
===== INVALID_GPA_REJECTION =====
```

```
-----  
Student Information Management System  
Version 4.0
```

```
Programmer: Branson Zemaitis
```

```
Welcome to SIMS  
-----
```

```
Enter Student ID: Enter Student Name: Enter Current GPA: ERROR  
Invalid GPA entered.  
GPA must be between 0.00 and 4.00.
```

GPA rounding edge case — 1.999 still rounds to 2.00 / Good Standing (my Part 3 fix still holds).

```

===== BOUNDARY_1999_ROUNDING =====
-----
Student Information Management System
Version 4.0

Programmer: Branson Zemaitis

Welcome to SIMS
-----

Enter Student ID: Enter Student Name: Enter Current GPA:
Enter Course 1 Grade: Enter Course 2 Grade: Enter Course 3 Grade: Enter Course 4 Grade: Enter Course 5 Grade:
Student Summary

Student ID : 5555555
Student Name : Boundary Test
Current GPA : 2.00
Academic Standing : Good Standing
-----
Course Grades

Course 1 : 50
Course 2 : 50
Course 3 : 50
Course 4 : 50
Course 5 : 50

-----
Average Grade : 50.00
Highest Grade : 50
Lowest Grade : 50

Student information successfully saved.

Reading student information...
File successfully loaded.

Recovered Student Record
-----

Student Information Management System
Version 4.0
-----

Student ID : 5555555
Student Name : Boundary Test
Current GPA : 2.00
Academic Standing : Good Standing
-----
Course Grades

Course 1 : 50
Course 2 : 50
Course 3 : 50
Course 4 : 50
Course 5 : 50

-----
Average Grade : 50.00
Highest Grade : 50
Lowest Grade : 50

-----
Verification: Recovered data matches the original data entered.

```

Regression Test Log

Required fields for each test, per the spec: description, input, expected output, actual output, pass/fail.

Test 1: Enter valid student information

Input: 1234567 / Alice Johnson / 3.84 / grades 95,88,91,84,100

Expected: Student report displayed correctly

Actual: Displayed correctly, Honors, avg 91.60, high 100, low 84 -- see screenshot on page 2.

Result: PASS

Test 2: Save information to file

Input: (continues Test 1's run)

Expected: File created successfully

Actual: "Student information successfully saved." printed -- see screenshot on page 2. File contents shown on page 5.

Result: PASS

Test 3: Read information from file

Input: (continues Test 1's run)

Expected: Data matches original input

Actual: "File successfully loaded." and Recovered Student Record printed -- see screenshot on page 4.

Result: PASS

Test 4: Compare original and recovered data

Input: (continues Test 1's run)

Expected: Information is identical

Actual: "Verification: Recovered data matches the original data entered." -- see screenshot on page 4.

Result: PASS

Test 5: Attempt to read a missing file

Input: Read permission revoked on student_records.txt after a successful save (see Assumption 2 for why this stands in for "missing")

Expected: Appropriate error message displayed

Actual: ERROR / Unable to open student_records.txt / Please verify that the file exists and that you have permission to access it. -- see screenshot on page 6, matches spec Example 3 exactly.

Result: PASS

Extra cases beyond the required five — Michael Brown, Christopher Williams, invalid GPA rejection, and the 1.999 rounding edge case (later in this document) — all produced their expected Part 2/3 output with no change, and all still passed verification after the save/reload round-trip.

Bugs I Found and Fixed

Bug 1: my program said the save worked even when it didn't. I wasn't checking whether `fclose` actually succeeded, so it always printed "Student information successfully saved" no matter what. `fprintf` only writes to a buffer — the real failure shows up when the file closes. I tested this by pointing the file at a device that always fails to flush: the program said it saved fine, then read back garbage from memory (Highest Grade : 1500501712) and exited like nothing was wrong. Fixed it by checking `fclose`'s return value and only claiming success when it's actually 0.

Bug 2: I wasn't checking if `scanf` actually worked when reading the file back. If a line wasn't a valid number, the variable was left uninitialized and the program just kept going with garbage. Now every read is checked, and a bad line gets called out by name — e.g. student_records.txt contains invalid data (Course 3 Grade).

Also fixed (not a bug): my write-failure and read-failure messages used different wording. The spec only gives one error message for "file cannot be opened" (Example 3), so both now print the same three lines.

Why I Refactored the Standing Text

I used to rebuild the Academic Standing text with its own switch statement everywhere I needed it. Part 4 needs it in three places now (on-screen, saved to the file, and verification), so I pulled it into one `standingText` variable set once and reused everywhere. Otherwise the three copies can drift apart — which is basically what caused my Part 3 GPA-rounding bug.

Answers to Questions for the Customer

1. What should the data file be named?

`student_records.txt` — it's literally the filename in the spec's own error example, so I just used it.

2. Should the program create the file if it doesn't exist?

Yes, automatically — opening in "w" mode does that on its own.

3. Overwrite or append to an existing file?

Overwrite. There's only one student this release, so there's nothing to append to.

4. What happens if the file can't be opened?

Print an error and exit — same message whether it's a read or a write failure.

5. Should it read the file back automatically after saving?

Yes, right away in the same run — that's what the spec's Example 1 shows.

6. Should the recovered report match the original exactly?

Yes, same fields and format, so I can compare them side by side.

7. Are course names saved too, or just grades?

Just grades — this program's never had course names, only five grade slots.

8. Is Academic Standing recalculated on read, or just read from the file?

Just read straight from the file as text, not recalculated.

9. What if the file has invalid or incomplete data?

Each line gets checked. If something's missing or not a number, it says exactly which field and exits.

10. Should the file have blank lines for readability?

No — kept it compact, one value per line, matching the spec's Example 2 exactly.

11. Will future versions store multiple students in one file?

Not this release — the spec says that's out of scope for now.

12. Should employees be able to choose the filename later?

Not yet — it's hardcoded for now.

Assumptions I Made

1. I treated the five required tests as the minimum, but also reran four extra regression cases from Parts 2 and 3 (Michael Brown, Christopher Williams, invalid GPA, the 1.999 rounding edge case) to prove nothing broke.

2. Test 5 can't literally happen the way it's worded — my program always writes the file right before reading it, so it's never actually "missing" at read time. The closest real equivalent is "can't be opened for reading,"

which I tested by revoking read permission. Same error message the spec asks for.

3. My recovered report shows Course 1 : 95 instead of the bare 95 in the spec's example. That's intentional — Question 6 says the recovered report should match my original report, and the Acceptance Criteria just says "neatly formatted," not "identical to the example."

4. There are two more error paths in my code beyond what's required — a failed save and bad data in the file — both described under Bugs above. I verified them by forcing the failure directly rather than through a normal run, since neither can happen in the program's normal flow. No screenshot for these since they're extra, not required.